
Benchmark Designers Should “Train on the Test Set” to Expose Exploitable Non-Visual Shortcuts

Ellis Brown Jihan Yang Shusheng Yang Rob Fergus Saining Xie

New York University

Abstract

Multimodal LLMs can answer many questions in vision-centric benchmarks without looking at the image, using linguistic priors and skewed answer distributions. Blind evaluation shows what a model already answers this way, but not which exploitable non-visual patterns the test set contains. Extending partial-input auditing (e.g., hypothesis-only baselines in natural language inference), we argue that benchmark designers should “train on the test set”: probe the artifact they release for exploitable patterns. Our Test-set Stress-Test (TsT) cross-validates a text-only Qwen2-7B on the test set’s questions and answer options, yielding a benchmark-level score and a per-question bias score $s(x)$; a random forest on hand-crafted features adds a fast, interpretable audit. On the template-based VSI-Bench and CV-Bench, the held-out score is 17.9 and 13.1 points above the model’s own zero-shot score. On MMMU it learns little, even though GPT-4o correctly answers 52.3% of its multiple-choice questions without the image, so blind success there reflects pretrained knowledge rather than learnable patterns in the test set. Iterative Bias Pruning (IBP) removes the questions with the highest $s(x)$ and re-diagnoses; on VSI-Bench it widens a fine-tuned model’s vision-blind gap more than random removal at the same rate. We also release VSI-Bench-Debiased, which lowers a fine-tuned model’s blind score from 44.7 to 32.0 while its vision score falls only from 57.1 to 48.7.

Project Page: <https://vision-x-nyu.github.io/test-set-training/>

1. Introduction

Benchmarks have driven progress in computer vision, from MNIST [28] and ImageNet [14] to COCO [31] and VQA [4]. As models grew more capable, benchmarks moved from narrow, controlled tasks to open-ended, real-world evaluation (Fig. 1). Multimodal LLMs (MLLMs) now let us ask anything about an image, but this flexibility has a cost: we have lost control over what is being measured.

A model can answer a visual question without looking at the image, using linguistic priors and skewed answer distributions. Language bias was already a concern in the VQA era (Sec. 5); LLMs, as both knowledge engines and sources of shortcuts, make it pressing again [39]. The usual safeguard is the “blind” test, which evaluates a multimodal benchmark with vision disabled [4, 9, 19, 39]. It shows what a model can already answer without looking. It does not show which questions become answerable once a model picks up non-visual patterns in the test set, for example from similarly generated training data, or how to fix them.

Benchmarking Visual Understanding

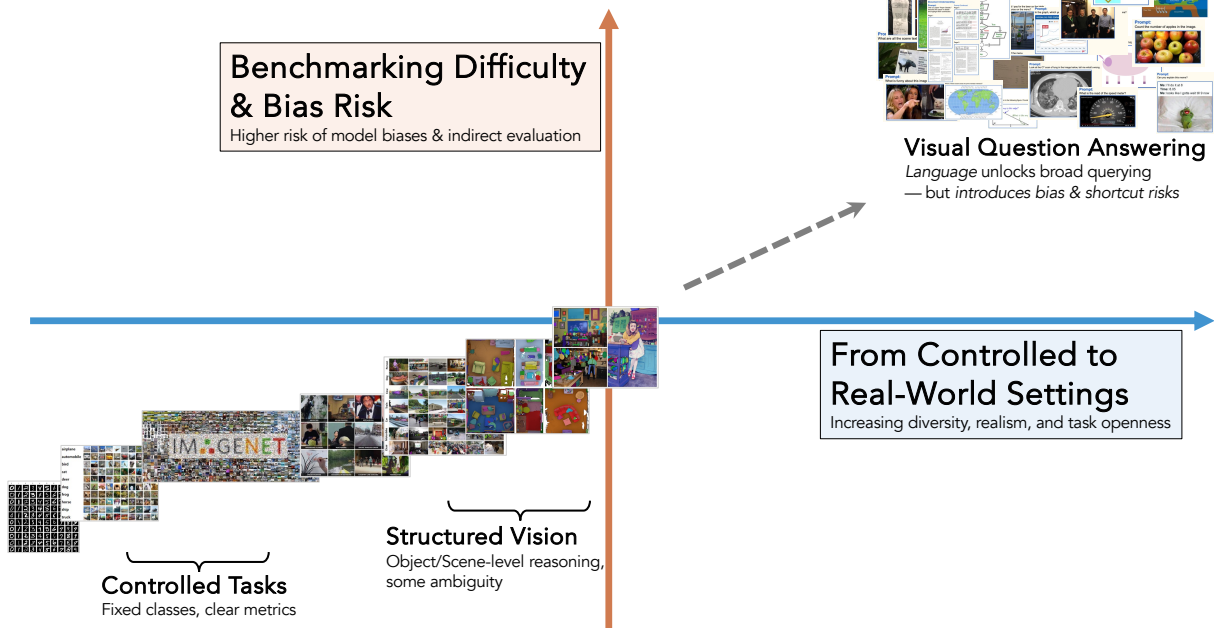


Figure 1 | **Visual benchmarks have moved from controlled tasks to open-ended VQA.** Asking questions in language makes benchmarks more expressive, but it also exposes them to non-visual shortcuts: models can score well by exploiting linguistic patterns without looking at the image.

If a benchmark *can* be gamed, it *will* be. We argue that **benchmark designers should “train on the test set”¹** to expose exploitable non-visual shortcuts: fit a probe to the test set’s non-visual input and measure what it learns. As in adversarial filtering [27, 47], benchmark design becomes iterative and adversarial.

The probe audits the data, not a model. Trained on a separate in-distribution split, where one exists, it finds biases of the process that generated the data. Cross-validated within the released test set, it can also catch *idiosyncratic vulnerabilities* of that specific artifact, such as those introduced by finite-sample selection, question templates, or human filtering.

Our Test-set Stress-Test (TsT) applies k -fold cross-validation to the test set’s non-visual input and yields a benchmark-level TsT score and a per-question bias score $s(x)$. TsT-LLM LoRA-tunes a language model on question and answer-option text; TsT-RF, a random forest on hand-crafted features, adds a fast, interpretable audit. Iterative Bias Pruning (IBP) then adapts AFLite’s iterative filtering [27]: it repeatedly removes the questions with the highest $s(x)$ and re-diagnoses the rest.

Learnable shortcuts vary widely across benchmarks (Sec. 3). On the template-based VSI-Bench [42] and CV-Bench [39], a blind Qwen2-7B cross-validated on the test set’s text scores 17.9 and 13.1 points above its own zero-shot score. On knowledge-intensive MMMU [45] it learns little, even though GPT-4o [24] correctly answers 52.3% of MMMU’s multiple-choice questions without the image; blind success there points mostly to pretrained knowledge.

We compare every pruning method with random removal at the same rate, on VSI-Bench and on MMMU, a non-templated negative control (Sec. 4). On VSI-Bench, a designer-in-the-loop pilot, built with hand-written per-type filters rather than by IBP, sharply lowers the blind score of a model fine-tuned on in-distribution data. It also widens the model’s vision-blind gap more than random removal does. Automated per-type IBP, run within each question type at the same removal rate, widens it less than the pilot but more than random

¹The phrasing is deliberately provocative and addressed to benchmark designers. We propose a held-out, pre-release audit of the benchmark itself, not a recipe for training deployed models on test answers.

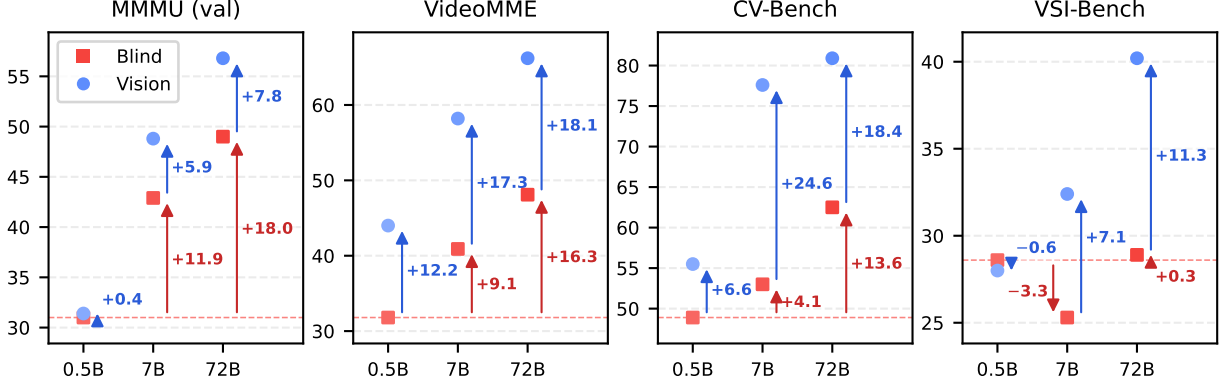


Figure 2 | **Blind scores rise with LLM scale on MMMU but not on VSI-Bench.** Blind ■ and vision-enabled ● scores of LLaVA-OneVision [29] at three LLM sizes. Red arrows give the blind gain over the smallest model’s blind score (dashed line); blue arrows, the gain from enabling vision. On MMMU, scaling the LLM adds more than enabling vision does; on VideoMME and CV-Bench, both help; on VSI-Bench, blind scores stay flat and the gains come from vision. VSI-Bench values are per-task averages from Yang et al. [42].

removal. On MMMU, pruning behaves like random removal.

Our contributions are: (1) a definition of an “exploitable” non-visual shortcut, with the designer deciding how to respond to flagged questions (Sec. 2); (2) the TsT diagnostic, with its LLM and random forest variants (Sec. 3); (3) diagnoses of VSI-Bench, CV-Bench, MMMU, and VideoMME, with a GPT-4o blind baseline (Sec. 3.4); (4) IBP, which adapts AFLite’s iterative filtering [27] to TsT scores, evaluated against random removal (Sec. 4); and (5) the VSI-Bench pilot, released as VSI-Bench-Debiased (Sec. 4.2).

2. The Challenge: Non-Visual Shortcuts

We define an **exploitable non-visual shortcut** as any pattern, regardless of origin, that renders visual input redundant for a task designed to measure visual understanding. Such shortcuts take two forms.

Knowledge-based shortcuts come from what an LLM learned in pretraining [39]: “How tall is the refrigerator?” can be answered from memorized appliance dimensions. On MMMU, blind scores rise with the size of the LLM backbone, consistent with such shortcuts; on VSI-Bench, they do not (Fig. 2).

Statistical shortcuts come from regularities in a benchmark’s questions and answers that a model can learn instead of visual reasoning (Fig. 3); they can be strong even where knowledge-based shortcuts are weak, as on VSI-Bench. Over half of VSI-Bench counting answers are “2”. In CV-Bench depth questions, books are the closer object in 60 of 64 questions, and a bookcase in none of 42. In VSI-Bench, objects favor particular positions in appearance-order questions, and room sizes are roughly log-normal.

Training data produced by the same generator carries the same regularities, so models can pick up these shortcuts without access to the test set. Fine-tuning LLaVA-Video-7B [50] on VSI-Train-10k, 10K in-distribution questions generated with VSI-Bench’s templates from training scenes (Appendix B), raises its *blind* VSI-Bench score by 18.8 points, nearly matching its 20.4-point gain with vision (Tab. 2, original columns). A benchmark’s exposure to such shortcuts is therefore worth measuring, for the benchmark as a whole and for each question.

TsT is meant for benchmark designers, before release. A high bias score $s(x)$ (Sec. 3.1) flags a question as exploitable; the designer decides whether to diversify the answer distribution, rewrite the question, or accept the bias.

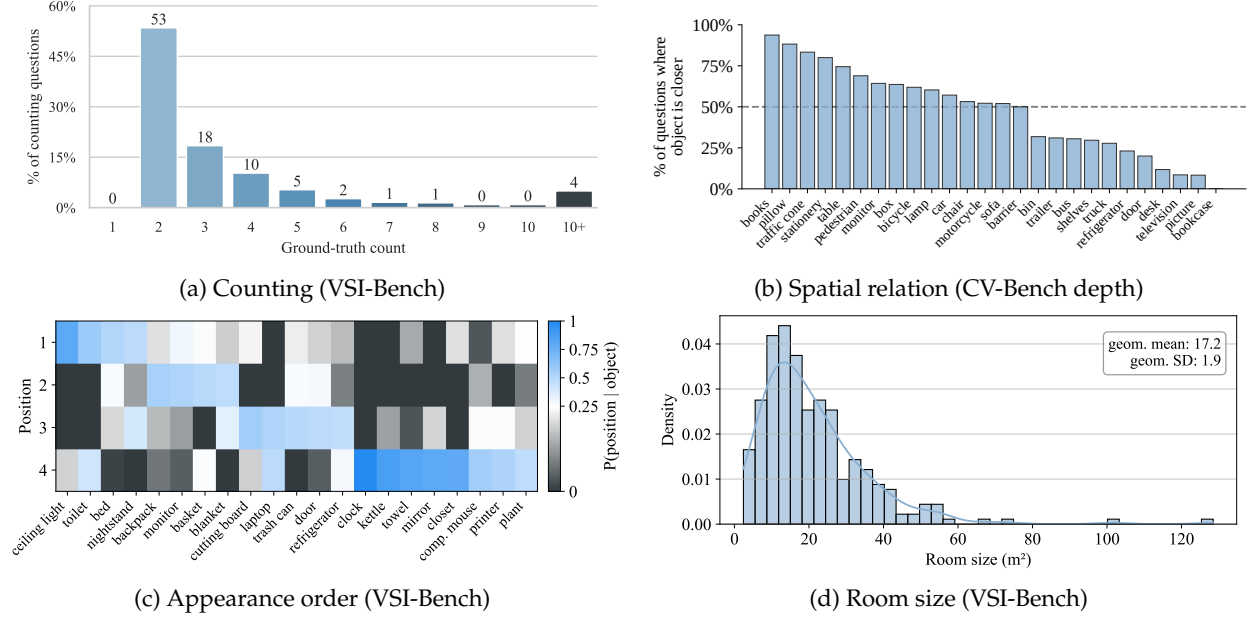


Figure 3 | **Statistical biases create non-visual shortcuts.** (a) Share of counting questions with each gold count. (b) How often each object is the closer one, for objects in at least 10 questions (dashed line: chance). (c) How often each object occupies each position of the gold appearance order, partly reflecting questions that share a video. (d) Room-size distribution, with geometric mean and geometric standard deviation.

3. Diagnosing Non-visual Shortcuts via Test-Set Stress-Testing

Benchmarks can fail in two ways: *training failures*, where biased training data prevents robust generalization (e.g., VQA-CP; 3), and *evaluation failures*, where the test set itself contains exploitable artifacts. TsT targets the latter. A diagnostic trained on separate in-distribution data finds biases of the data-generating process but can miss idiosyncratic vulnerabilities of a particular test set (Fig. 4a). We therefore probe the test set itself, which is also the practical choice: most MLLM benchmarks are released with only an evaluation split.

3.1. The TsT Framework

TsT applies k -fold cross-validation [21, 37] to the test set: split it into k disjoint folds (typically $k = 5$), train a blind diagnostic on the non-visual content of $k-1$ folds, and score the held-out fold (Fig. 4b). After k rounds, every question has been scored by a model that never trained on it. Folds are drawn at random over questions; Tab. 3 and the Limitations section also report folds grouped by scene or image.

For each question, the diagnostic’s score is the probability it assigns to the gold option for multiple-choice (MC) questions (for TsT-LLM, renormalized over the option letters) and the mean relative accuracy (MRA) of its generated answer for numerical (NUM) questions. TsT yields two outputs:

1. **TsT score:** the mean held-out score over all questions, an estimate of the benchmark’s non-visual solvability. It is a lower bound on what a stronger diagnostic could exploit with the same access. It is also a designer-side stress test, not ordinary model performance: the diagnostic trains on gold answers from the other folds, which a model under evaluation never sees.
2. **Bias score $s(x)$:** the held-out score of question x . A high $s(x)$ flags x as likely answerable without visual input and a candidate for pruning (Sec. 4).

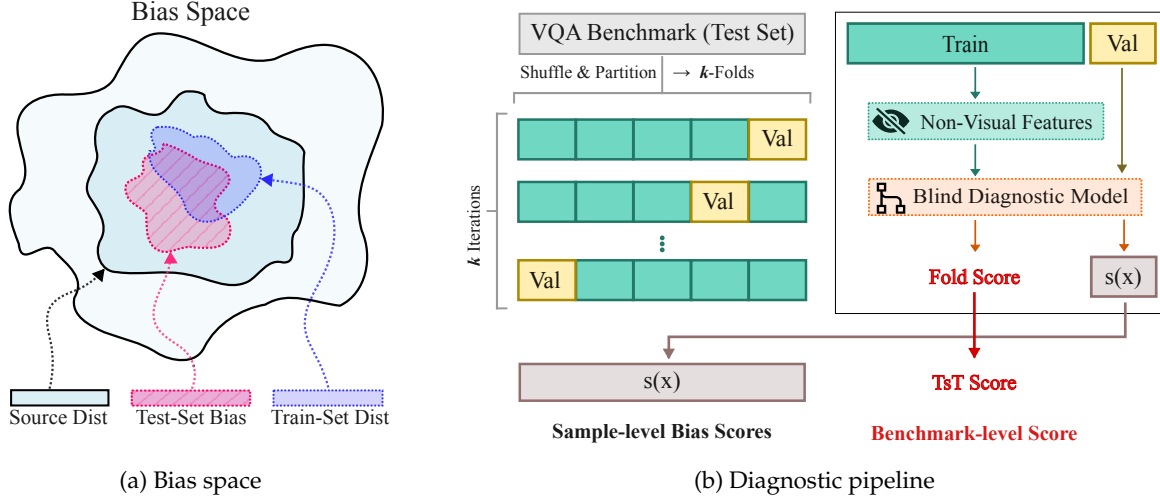


Figure 4 | **TsT probes the test set directly.** (a) A diagnostic trained on separate data from the source distribution (dotted blue) finds only the test-set biases (pink) that its training data shares and can miss those specific to the test set. (b) The test set is split into k folds; a blind diagnostic trains on $k-1$ folds and scores the held-out fold, k times over, giving a benchmark-level TsT score and a per-question bias score $s(x)$.

3.2. LLM-Based TsT Diagnostic

Our primary diagnostic fine-tunes a language model (Qwen2-7B; 41) with LoRA [23] on the question and answer-option text of the training folds and scores the held-out fold. It needs no feature engineering. Comparing its TsT score with the same model’s zero-shot (ZS) score separates pretrained knowledge from what it learns from the test set. Δ_{TsT} is the TsT score minus the ZS score, and $\Delta s(x) = s(x) - s_{\text{ZS}}(x)$ is its per-question counterpart. Ranking by $s(x)$ also surfaces questions answerable from pretrained knowledge; ranking by $\Delta s(x)$ targets what is learnable from the test set itself. The full $k=5$ procedure takes about 36 minutes for VSI-Bench (5,130 questions) and 14 minutes for MMMU on one H200 GPU (Appendix C.1).

3.3. RF-Based TsT Diagnostic

TsT-RF, a secondary diagnostic for template-based benchmarks, is a random forest [7] on hand-crafted non-visual features such as question structure, answer properties, and task-specific statistics (Appendix C.2). Its benchmark-level MC score is top-1 accuracy; it has no zero-shot counterpart, so it ranks questions by $s(x)$. TsT-RF trades generality for interpretability and efficiency: it trains in seconds on a CPU, and its feature importances show which cues drive it. On VSI-Bench object size estimation, the object category’s mean log size alone has Gini importance 0.968: categories such as dishwashers barely vary in size, so designers can drop their questions or sample more varied instances (Appendix E). Most of its features are *label-conditioned*: statistics of gold answers in the training folds, such as a category’s mean size; none of those we use is keyed on the held-out question’s own answer. Its inputs, not only its targets, carry gold-answer information that a model under evaluation never sees (Appendix C.2), so we treat TsT-RF scores as designer-side audit statistics, not as blind-model performance.

3.4. Empirical Validation: Shortcut Susceptibility Varies Across Benchmarks

TsT-LLM Results. Tab. 1 compares a blind Qwen2-7B’s ZS score with its TsT score on VSI-Bench, CV-Bench, VideoMME [17], and MMMU, with one configuration for every benchmark (Appendix C.1). On the template-based VSI-Bench and CV-Bench, TsT-LLM gains 17.9 and 13.1 points over zero-shot, so much of each benchmark can be answered by learning patterns in its non-visual text alone. VideoMME gains little (+4.1). On the knowledge-intensive, non-templated MMMU, fine-tuning changes the score little ($\Delta_{\text{TsT}} = -0.7$ by gold-option probability; +5.4 by top-1 accuracy, about the gain on MMStar below). GPT-4o [24], however,

Table 1 | **Learnable non-visual shortcuts are large on template-based benchmarks and small elsewhere.** A blind Qwen2-7B is scored zero-shot (ZS) and after k -fold LoRA fine-tuning on the other folds of the test set (TsT), with the same rule (Appendix C.1): mean gold-option probability for MC and MRA for NUM questions, averaged over questions; VSI-Bench (MC+NUM) weights the two formats by question count. Δ_{TsT} is TsT minus ZS, computed from unrounded scores. GPT-4o blind top-1 accuracy is shown for context and is not directly comparable with the Qwen columns.

Benchmark	Qwen2-7B, blind			GPT-4o, blind top-1 acc.
	ZS	TsT	Δ_{TsT}	
CV-Bench	47.3	60.3	+13.1*	44.8
VSI-Bench (MC)	25.5	39.1	+13.6	34.0
VSI-Bench (NUM)	24.0	46.0	+21.9	–
VSI-Bench (MC+NUM)	24.7	42.7	+17.9	–
VideoMME	29.2	33.4	+4.1	46.6
MMMU (val, MC)	34.2	33.6	−0.7	52.3
MMStar (control)	24.2	29.5	+5.4	–

*Scoring zero-shot non-answers as wrong gives +18.4 (Appendix C.1).

correctly answers 52.3% of its MC questions without the image.² Blind success on MMMU therefore points mostly to pretrained knowledge, not to learnable patterns in the test set.

Vision-indispensable control. On MMStar [9], curated to require vision, Qwen2-7B’s blind zero-shot score (24.2) is close to the 25% uniform baseline and to its score on VSI-Bench’s MC questions (25.5). Yet TsT adds only 5.4 points on MMStar, against 13.6 on VSI-Bench’s MC questions. TsT reports no large learnable shortcuts where few are expected, and separates two benchmarks that a blind zero-shot test scores alike.

TsT-RF Results. TsT-RF reaches 43.0% on VSI-Bench, against its 34.0% per-task frequency baseline, and 56.1% on CV-Bench, against 41.8% uniform chance (Tab. 3, which also reports grouped folds). At the benchmark level it points the same way as TsT-LLM.

4. Mitigating Non-Visual Shortcuts Guided by TsT Insights

We evaluate pruning by its effect on an MLLM’s blind score and on its vision-blind gap, its score with vision minus its score without. We compare every pruning method with random removal at the same rate.

4.1. Iterative Bias Pruning (IBP)

IBP removes the questions with the highest $s(x)$ in batches and re-diagnoses the rest after each batch (Algorithm 1), following AFLite [27], whose ablation found iterative filtering more effective than a single pass. We re-diagnose since removing questions changes which of the remaining ones are predictable and can create new artifacts [20]. A removal budget B trades bias reduction against coverage, the batch size b sets how often to re-diagnose, and a threshold τ on the largest remaining score stops the procedure early. Algorithm 1 accepts any per-question score (Appendix D.1); our IBP runs rank by TsT-RF $s(x)$ on VSI-Bench (Tab. 10) and by TsT-LLM $\Delta s(x)$ on MMMU (Tab. 6).

4.2. Pruning VSI-Bench

VSI-Bench-Debiased is a designer-in-the-loop pilot, not the output of Algorithm 1. Hand-written filters for each question type remove 2,768 of 5,130 questions (54.0%). They use only answer statistics, such as how

²GPT-4o (gpt-4o-2024-08-06, temperature 0, at most 10 output tokens, letter-labeled options) is scored by top-1 accuracy on MC questions (all 2,490 on VSI-Bench; 847 on MMMU); responses with no parseable option, mostly refusals, count as incorrect.

Algorithm 1: Iterative Bias Pruning (IBP)

Input: Dataset $\mathcal{D}$; bias-diagnosis function $\text{ComputeSampleBiasScores}(\cdot)$; removal budget B ; batch size b ; early-stopping threshold τ
Output: Debiased dataset $\mathcal{D}'$

```
 $R \leftarrow 0$  ; // samples removed so far
while  $R < B$  do
     $\{s_i\}_{x_i \in \mathcal{D}} \leftarrow \text{ComputeSampleBiasScores}(\mathcal{D})$  ; // re-diagnose
    if  $\max_i s_i \leq \tau$  then
        break ; // early stop: all biases below threshold
     $m \leftarrow \min(b, B - R)$ ;
     $\mathcal{I} \leftarrow \text{SelectBatch}(\{s_i\}_{x_i \in \mathcal{D}}, m, \mathcal{D})$  ; // select batch for removal
     $\mathcal{D} \leftarrow \mathcal{D} \setminus \mathcal{I}$ ;
     $R \leftarrow R + |\mathcal{I}|$ ;
return  $\mathcal{D}' \leftarrow \mathcal{D}$ 
```

Table 2 | **VSI-Bench-Debiased (designer-in-the-loop pilot)**. LLaVA-Video-7B scores (MC accuracy and NUM MRA, averaged over questions) with vision (Vis.) and without (Blind), before and after fine-tuning on VSI-Train-10k; Gap is Vis. minus Blind. On the original set, fine-tuning raises vision and blind scores almost equally (+20.4 vs. +18.8); on the debiased set it raises vision much more than blind (+17.4 vs. +11.7), so more of the gain there requires vision.

Model	VSI-Bench (Original)			VSI-Bench-Debiased		
	Vis.	Blind	Gap	Vis.	Blind	Gap
LLaVA-Video-7B (base)	36.7	25.9	10.8	31.3	20.3	11.0
+ VSI-Train-10k FT	57.1	44.7	12.4	48.7	32.0	16.7
<i>Gain from FT</i>	+20.4	+18.8	+1.6	+17.4	+11.7	+5.7

close an object’s size is to its category mean, the cue that dominates TsT-RF on object size (Sec. 3.3), or how often an option or object pair is the answer.

The pilot preferentially removes blind-exploitable questions, so the drop in blind score is the primary signal. For LLaVA-Video-7B fine-tuned on VSI-Train-10k, the blind score falls from 44.7 to 32.0, while the vision score falls less, from 57.1 to 48.7 (Tab. 2). The vision-blind gap thus widens from 12.4 to 16.7, whereas removing 54% of questions at random leaves it at 12.4 on average (Tab. 8). For Cambrian-S [43], a video MLLM for spatial reasoning, the pilot widens the gap from 19.2 to 26.0, against 19.4 for random removal. The widening therefore depends on which questions are removed. Its size varies by model: it is large only for these two, which have the highest original blind scores (Tab. 9), so we treat it as secondary.

Automated per-type IBP on TsT-RF scores widens the gap less than the pilot but more than random removal (Tab. 10). It runs within each question type, with uniform per-type budgets (the same fraction from every type) or the pilot’s per-type budgets (its removal count for each type). At the pilot’s 54% removal rate, per-type IBP reaches gaps of 14.1 to 14.7 across the two budget schemes, against 12.2 to 12.4 for random removal of the same number of questions, and 16.7 for the pilot. Per-format IBP, which ranks MC and NUM questions separately, removes 19.5% of questions at $B=1000$ and leaves the gap at 12.4, close to random removal at that rate (12.2).

VSI-Bench-Debiased keeps only 2,362 questions and changes the question-type mix: it removes 85.7% of medium but only 2.3% of easy relative-direction questions (Tab. 7). We therefore release it as a separate evaluation artifact, not a drop-in replacement for VSI-Bench.

4.3. Pruning MMMU

We apply IBP with TsT-LLM to the 900-question MMMU validation set [45] as a negative control, since TsT finds few learnable shortcuts there (Sec. 3.4). Ranking its 847 MC questions by $\Delta s(x)$, we remove 100 (11.1% of the set) and keep all 53 open-ended questions, which TsT-LLM does not score.

On the remaining 800 questions, LLaVA-OneVision-7B’s gap moves only from 6.7 to 7.5, inside the range of 1,000 matched random removals (4.62 to 8.75; Tab. 6). The removed questions are mostly general-knowledge questions, consistent with MMMU’s near-zero Δ_{TsT} . The control is weak, however: the same model’s gap also barely changes on VSI-Bench-Debiased (−0.5; Tab. 9).

5. Related Work

Partial-input and blind tests. Scoring a benchmark from part of its input is a long-standing check. VQA reported question-only baselines [4], VQA v2 was compared against a language-only model [19], and on multiple-choice Visual7W, models given only the question and candidate answers were competitive with models that also saw the image [25]. In NLI, hypothesis-only classifiers predict the label well above the majority class [20, 34]. For MLLMs, the same check is the “blind” test, which MMStar and Cambrian-1 ran across many benchmarks [9, 39]. Related diagnostics score per-question modality importance [33], reliance on local versus global image content [2], and unimodal bias [10]. TsT is also a partial-input probe, with the image as the omitted input, but it measures something different. A blind test asks what a prompted model answers without the image; TsT asks how much more a text-only model answers after training on the other folds of the same test set. The two can disagree: blind zero-shot scores on VSI-Bench’s MC questions and on MMStar, which was already filtered with blind LLMs, are similar, but their TsT gains differ widely (Sec. 3.4).

Filtering benchmark items. These probes have also been used to revise benchmarks. Gururangan et al. [20] used a hypothesis-only classifier, trained on the training split, to split NLI test sets into Easy and Hard subsets. MLLM benchmarks use the blind test itself as a filter. MMStar excluded candidates that LLMs answered correctly without the image, then chose its final set by manual review [9]; MMMU-Pro begins by removing MMMU questions that text-only LLMs answer correctly [46]. Adversarial filtering in SWAG and HellaSwag replaces machine-generated distractors that discriminators reject easily [47, 48]. AFLite [27], used to build WinoGrande [36], instead removes items: it repeatedly drops those that classifiers trained on other partitions predict most reliably, and compares the result with size-matched random subsets. IBP (Algorithm 1) follows AFLite’s loop and its random-removal control. Partial-input probes, cross-validated per-item scores, and iterative filtering all predate this work; what differs is the probe. AFLite’s classifiers usually see the full input, whereas ours sees only the non-visual input and is fit within the released test set. TsT-LLM also subtracts the same model’s zero-shot score, so Δ_{TsT} separates what a pretrained model already knows from what it learns from the test set.

Debiasing VQA. Some VQA work changed the evaluation data. VQA v2 pairs questions with complementary images that lead to different answers [19], and VQA-CP re-splits VQA so that answer priors differ between training and test [3]. Later work mines counterexample subsets [13] or synthesizes evaluation data to remove answer-option biases [40]. Most work instead takes the benchmark as given and changes the model. Approaches include grounded architectures [3], adversarial regularization against a question-only branch [35], re-weighting examples that a question-only branch answers [8, 30], counterfactual inference or training [1, 32], and generative bias models [11]. Belinkov et al. [6] remove hypothesis-only bias from NLI models with adversarial training. TsT is dataset-side work. Its probe resembles these question-only branches, but it is fit on the released test set, and its scores revise the benchmark rather than train a less biased model.

Robust benchmarks and shortcut learning. Winoground [38], Vinoground [49], and the Hateful Memes Challenge [26] were designed from the start to resist shortcuts, and a broader literature studies why models prefer available but non-causal features [15, 18, 22]. Visual shortcuts matter too; we focus on non-visual ones because they violate the stated intent of a vision-centric benchmark.

Limitations

Fold construction. Folds split questions, not the scenes or images behind them, so random folds also credit regularities shared by questions about one scene. Grouping VSI-Bench folds by scene lowers TsT-RF from 43.0 to 38.8 (Tab. 3), mostly in appearance order; object-size and counting priors remain; the TsT-LLM gain changes little (17.9 to 17.0 points). Designers should report both random- and grouped-fold scores.

Remaining shortcuts. VSI-Bench-Debiased reduces non-visual shortcuts without removing them, and pruning can create new answer priors. We did not re-run TsT on the pruned set; any refined version should be re-diagnosed with grouped folds before release.

Null results. A small Δ_{TsT} , as on MMMU, is only a lower bound on learnable non-visual signal. A stronger diagnostic may find more, and no partial-input probe detects artifacts involving the omitted image [16].

Dependence on the diagnostic. TsT scores depend on the diagnostic’s configuration: on CV-Bench, the scoring rule and rerun variation each move Δ_{TsT} by several points (Appendix C.1). TsT-LLM and TsT-RF select largely different questions for removal (Tab. 12), and rankings change substantially across base models (Appendix D.5). TsT-RF needs hand-crafted features for each benchmark, and we have not reviewed flagged questions systematically; Appendix E shows examples.

6. Conclusion

Models can score well on multimodal benchmarks without using the image, so benchmark scores can overstate visual understanding. We argue that benchmark designers should “train on the test set” before release. TsT cross-validates a blind model on the test set’s text, yielding a benchmark-level TsT score and a per-question bias score $s(x)$. Learnable shortcuts vary widely. On the template-based VSI-Bench and CV-Bench, TsT-LLM lifts a blind Qwen2-7B 17.9 and 13.1 points above its zero-shot score; on knowledge-intensive MMMU it learns little, although GPT-4o correctly answers 52.3% of MMMU’s multiple-choice questions without the image. On VSI-Bench, our designer-in-the-loop pilot, released as VSI-Bench-Debiased, lowers a fine-tuned model’s blind score from 44.7 to 32.0 and widens its vision-blind gap more than random removal at the same rate. Automated per-type IBP widens it less, though still more than random removal. On MMMU, pruning by $\Delta s(x)$ behaves like random removal.

Open directions include other modalities, open-ended answers, transfer across diagnostics, and rewriting questions or rebalancing answers instead of removing them. We release VSI-Bench-Debiased on the project page and our code at <https://github.com/vision-x-nyu/test-set-training>. Test-set stress-testing should become a standard step in multimodal benchmark design.

Acknowledgments

We thank Shengbang Tong and Anjali Gupta for reviewing this manuscript, Chris Hoang for helpful discussions, and Zaid Khan for advice on LoRA frameworks. E.B. is supported by the DoD NDSEG Fellowship Program, and S.X. by the MSIT IITP grant (RS-2024-00457882) and NSF award IIS-2443404.

References

- [1] Ehsan Abbasnejad, Damien Teney, Amin Parvaneh, Javen Shi, and Anton van den Hengel. Counterfactual vision and language learning. In *CVPR*, 2020.
- [2] Amit Agarwal, Hitesh Laxmichand Patel, Srikant Panda, Hansa Meghwani, Jyotika Singh, Karan Dua, Paul Li, Tao Sheng, Sujith Ravi, and Dan Roth. RCI: A score for evaluating global and local reasoning in multimodal benchmarks. In *EMNLP (Industry Track)*, pages 138–157, 2025. doi: 10.18653/v1/2025.emnlp-industry.10.
- [3] Aishwarya Agrawal, Dhruv Batra, Devi Parikh, and Aniruddha Kembhavi. Don’t just assume; look and answer: Overcoming priors for visual question answering. In *CVPR*, 2018.
- [4] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh. VQA: Visual Question Answering. In *ICCV*, 2015.
- [5] Gilad Baruch, Zhuoyuan Chen, Afshin Dehghan, Tal Dimry, Yuri Feigin, Peter Fu, Thomas Gebauer, Brandon Joffe, Daniel Kurz, Arik Schwartz, and Elad Shulman. ARKitscenes - a diverse real-world dataset for 3d indoor scene understanding using mobile RGB-d data. In *NeurIPS*, 2021.
- [6] Yonatan Belinkov, Adam Poliak, Stuart Shieber, Benjamin Van Durme, and Alexander Rush. On adversarial removal of hypothesis-only bias in natural language inference. In **SEM*, pages 256–262. Association for Computational Linguistics, 2019. doi: 10.18653/v1/S19-1028. URL <https://aclanthology.org/S19-1028/>.
- [7] Leo Breiman. Random forests. *Machine learning*, 2001.
- [8] Remi Cadene, Corentin Dancette, Hedi Ben-younes, Matthieu Cord, and Devi Parikh. RUBi: Reducing unimodal biases for visual question answering. *NeurIPS*, 2019.
- [9] Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Zehui Chen, Haodong Duan, Jiaqi Wang, Yu Qiao, Dahua Lin, and Feng Zhao. Are we on the right way for evaluating large vision-language models? In *NeurIPS*, pages 27056–27087, 2024. doi: 10.52202/079017-0850.
- [10] Meiqi Chen, Yixin Cao, Yan Zhang, and Chaochao Lu. Quantifying and mitigating unimodal biases in multimodal large language models: A causal perspective. In *Findings of EMNLP*, pages 16449–16469, 2024. doi: 10.18653/v1/2024.findings-emnlp.960.
- [11] Jae Won Cho, Dong-Jin Kim, Hyeonggon Ryu, and In So Kweon. Generative bias for robust visual question answering. In *CVPR*, 2023.
- [12] Angela Dai, Angel X Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes. In *CVPR*, 2017.
- [13] Corentin Dancette, Remi Cadene, Damien Teney, and Matthieu Cord. Beyond question-based biases: Assessing multimodal shortcut learning in visual question answering. In *ICCV*, pages 1554–1563, 2021. doi: 10.1109/ICCV48922.2021.00160.
- [14] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009.
- [15] Mengnan Du, Varun Manjunatha, Rajiv Jain, Ruchi Deshpande, Franck Dernoncourt, Jiuxiang Gu, Tong Sun, and Xia Hu. Towards interpreting and mitigating shortcut learning behavior of NLU models. In *NAACL*, pages 915–929, 2021. doi: 10.18653/v1/2021.naacl-main.71.
- [16] Shi Feng, Eric Wallace, and Jordan Boyd-Graber. Misleading failures of partial-input baselines. In *ACL*, pages 5533–5538. Association for Computational Linguistics, 2019. doi: 10.18653/v1/P19-1554. URL <https://aclanthology.org/P19-1554/>.

- [17] Chaoyou Fu, Yuhang Dai, Yongdong Luo, Lei Li, Shuhuai Ren, Renrui Zhang, Zihan Wang, Chenyu Zhou, Yunhang Shen, Mengdan Zhang, et al. Video-MME: The first-ever comprehensive evaluation benchmark of multi-modal LLMs in video analysis. In *CVPR*, 2025.
- [18] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. doi: 10.1038/s42256-020-00257-z.
- [19] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. Making the V in VQA matter: Elevating the role of image understanding in visual question answering. In *CVPR*, 2017.
- [20] Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *NAACL*, pages 107–112. Association for Computational Linguistics, 2018. doi: 10.18653/v1/N18-2017. URL <https://aclanthology.org/N18-2017/>.
- [21] Trevor Hastie, Robert Tibshirani, and Jerome H. Friedman. *The elements of statistical learning: data mining, inference, and prediction*. Springer, 2 edition, 2009. doi: 10.1007/978-0-387-84858-7.
- [22] Katherine L Hermann, Hossein Mobahi, Thomas Fel, and Michael C Mozer. On the foundations of shortcut learning. In *ICLR*, 2024. URL <https://arxiv.org/abs/2310.16228>.
- [23] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*, 2022.
- [24] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. GPT-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- [25] Allan Jabri, Armand Joulin, and Laurens van der Maaten. Revisiting visual question answering baselines. In *ECCV*, pages 727–739, 2016. doi: 10.1007/978-3-319-46484-8_44.
- [26] Douwe Kiela, Hamed Firooz, Aravind Mohan, Vedanuj Goswami, Amanpreet Singh, Pratik Ringshia, and Davide Testuggine. The hateful memes challenge: Detecting hate speech in multimodal memes. In *NeurIPS*, 2020.
- [27] Ronan Le Bras, Swabha Swayamdipta, Chandra Bhagavatula, Rowan Zellers, Matthew Peters, Ashish Sabharwal, and Yejin Choi. Adversarial filters of dataset biases. In *ICML*, volume 119 of *Proceedings of Machine Learning Research*, pages 1078–1088, 2020. URL <https://proceedings.mlr.press/v119/bras20a.html>.
- [28] Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 1998.
- [29] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. LLaVA-OneVision: Easy visual task transfer. *Transactions on Machine Learning Research*, 2025.
- [30] Zujie Liang, Haifeng Hu, and Jiaying Zhu. LPF: A language-prior feedback objective function for de-biased visual question answering. In *SIGIR*, 2021.
- [31] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *ECCV*, 2014.
- [32] Yulei Niu, Kaihua Tang, Hanwang Zhang, Zhiwu Lu, Xian-Sheng Hua, and Ji-Rong Wen. Counterfactual VQA: A cause-effect look at language bias. In *CVPR*, 2021.
- [33] Jean Park, Kuk Jin Jang, Basam Alasaly, Sriharsha Mopidevi, Andrew Zolensky, Eric Eaton, Insup Lee, and Kevin Johnson. Assessing modality bias in video question answering benchmarks with multimodal large language models. In *AAAI*, 2025.

- [34] Adam Poliak, Jason Naradowsky, Aparajita Haldar, Rachel Rudinger, and Benjamin Van Durme. Hypothesis only baselines in natural language inference. In **SEM*, pages 180–191. Association for Computational Linguistics, 2018. doi: 10.18653/v1/S18-2023. URL <https://aclanthology.org/S18-2023/>.
- [35] Sainandan Ramakrishnan, Aishwarya Agrawal, and Stefan Lee. Overcoming language priors in visual question answering with adversarial regularization. In *NeurIPS*, 2018.
- [36] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial Winograd schema challenge at scale. In *AAAI*, pages 8732–8740, 2020. doi: 10.1609/aaai.v34i05.6399.
- [37] Mervyn Stone. Cross-validators choice and assessment of statistical predictions. *Journal of the royal statistical society*, 1974.
- [38] Tristan Thrush, Ryan Jiang, Max Bartolo, Amanpreet Singh, Adina Williams, Douwe Kiela, and Candace Ross. Winoground: Probing vision and language models for visio-linguistic compositionality. In *CVPR*, 2022.
- [39] Shengbang Tong, Ellis Brown, Penghao Wu, Sanghyun Woo, Manoj Middepogu, Sai Charitha Akula, Jihan Yang, Shusheng Yang, Adithya Iyer, Xichen Pan, Austin Wang, Rob Fergus, Yann LeCun, and Saining Xie. Cambrian-1: A Fully Open, Vision-Centric Exploration of Multimodal LLMs. *NeurIPS*, 2024.
- [40] Zhecan Wang, Long Chen, Haoxuan You, Keyang Xu, Yicheng He, Wenhao Li, Noel Codella, Kai-Wei Chang, and Shih-Fu Chang. Dataset bias mitigation in multiple-choice visual question answering and beyond. In *Findings of EMNLP*, pages 8598–8617. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-emnlp.576. URL <https://aclanthology.org/2023.findings-emnlp.576/>.
- [41] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024. doi: 10.48550/arXiv.2407.10671. URL <https://arxiv.org/abs/2407.10671>.
- [42] Jihan Yang, Shusheng Yang, Anjali W. Gupta, Rilyn Han, Li Fei-Fei, and Saining Xie. Thinking in Space: How Multimodal Large Language Models See, Remember and Recall Spaces. *CVPR*, 2025.
- [43] Shusheng Yang, Jihan Yang, Pinzhi Huang, Ellis Brown, Zihao Yang, Yue Yu, Shengbang Tong, Zihan Zheng, Yifan Xu, Muhan Wang, Daohan Lu, Rob Fergus, Yann LeCun, Li Fei-Fei, and Saining Xie. Cambrian-S: Towards Spatial Supersensing in Video. In *ICLR*, 2026.
- [44] Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *ICCV*, 2023.
- [45] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, Huan Sun, Yu Su, and Wenhui Chen. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *CVPR*, 2024.
- [46] Xiang Yue, Tianyu Zheng, Yuansheng Ni, Yubo Wang, Kai Zhang, Shengbang Tong, Yuxuan Sun, Botao Yu, Ge Zhang, Huan Sun, Yu Su, Wenhui Chen, and Graham Neubig. MMMU-Pro: A more robust multi-discipline multimodal understanding benchmark. In *ACL*, pages 15134–15186, 2025. doi: 10.18653/v1/2025.acl-long.736.
- [47] Rowan Zellers, Yonatan Bisk, Roy Schwartz, and Yejin Choi. SWAG: A large-scale adversarial dataset for grounded commonsense inference. In *EMNLP*, pages 93–104. Association for Computational Linguistics, 2018. doi: 10.18653/v1/D18-1009. URL <https://aclanthology.org/D18-1009/>.

- [48] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *ACL*, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/P19-1472. URL <https://aclanthology.org/P19-1472/>.
- [49] Jianrui Zhang, Mu Cai, and Yong Jae Lee. Vinoground: Scrutinizing LMMs over dense temporal reasoning with short videos. *arXiv preprint arXiv:2410.02763*, 2024.
- [50] Yuanhan Zhang, Jinming Wu, Wei Li, Bo Li, Zejun Ma, Ziwei Liu, and Chunyuan Li. LLaVA-Video: Video instruction tuning with synthetic data. *Transactions on Machine Learning Research*, 2025.

Appendix

This appendix gives the LLM-use disclosure (§A), VSI-Train-10k generation (§B), TsT diagnostic details (§C), debiasing details (§D), and the TsT-RF interpretability case study (§E).

A. LLM Disclosure

LLMs were used for drafting and editing assistance, bibliographic and consistency checks, coding assistance, and data visualization in preparing this manuscript. The authors directed the work, verified the reported claims, citations, and numerical results against the sources, and take responsibility for the final text. Research ideas, experimental design, data generation, and evaluation were done and validated by the authors.

B. VSI-Train-10k Generation

VSI-Train-10k is the in-distribution training set used to fine-tune LLaVA-Video-7B in Tab. 2. We build it with the VSI-Bench curation pipeline [42, 43], applied to the training splits of ScanNet [12], ScanNet++ [44], and ARKitScenes [5]. We extract object counts, bounding boxes, and room sizes, generate questions with VSI-Bench’s templates for its seven rule-based question types (all but route planning), and randomly sample 10K questions, with at most 20 per question type per scene.

C. TsT Diagnostic Details

C.1. TsT-LLM Implementation Details

Every TsT-LLM result in Tab. 1 uses Qwen2-7B-Instruct [41] (revision f2826a0; datasets VSI-Bench d7cb1a3, CV-Bench bc284db, VideoMME ead1408) with one configuration. Cross-validation: $k=5$, seed 42, one repeat. LoRA [23]: rank $r=8$, $\alpha=16$, dropout 0.1, all-module targets. Training: one epoch per fold, train and evaluation batch size 4, gradient accumulation 1, learning rate 2×10^{-4} with cosine decay and 0.1 warmup, FP16, maximum sequence length 1024, a 10% validation split of the training folds, and the gemma training template. Inference: raw prompts (no chat template), temperature 0, top- p 1, at most 10 new tokens, and top-100 log probabilities.

Each question is scored only in its held-out fold: an MC question by the gold option’s probability, a NUM question by the MRA of its generated answer. If the gold letter is not among the top-100 log probabilities, we parse the generated answer instead; an answer with no parseable option receives a random option. This fallback does not fire after fine-tuning and has little effect on VSI-Bench and VideoMME. On CV-Bench, however, many zero-shot answers begin with a word rather than an option letter and so receive chance credit. Scoring them as wrong would give a CV-Bench Δ_{TsT} of about +18.4 rather than +13.1; we report the smaller value, since the larger one would credit TsT for learning the answer format.

LoRA training on GPUs is not bit-reproducible. Reruns of this configuration give VSI-Bench (MC+NUM) scores of 42.6–42.8 ($\Delta_{\text{TsT}} \approx 18$); the CV-Bench gain varies more across reruns and fold assignments, from about 13 to 18 points.

We also ran a separate chat-formatted configuration ($r=32$, $\alpha=64$, two epochs, batch 32, learning rate 5×10^{-5} , cosine decay, 0.1 warmup, and the qwen chat template). Its $\Delta s(x)$ rankings agree with a matched $r=128$, $\alpha=256$, three-epoch run of the same setup (Spearman correlation 0.87; 70.9% top-1,000 overlap).

C.2. TsT-RF Features

TsT-RF (Sec. 3.3) uses non-visual features tailored to each benchmark’s question templates. They fall into four groups: (1) entities and relations parsed from the question text, such as object categories, distances, and directions; (2) answer-space properties, such as answer format and option count; (3) question-type and object metadata; and (4) task-specific frequency and distributional statistics fit on the training folds, such as

the average size of an object category for size estimation. For VSI-Bench, regular expressions parse these features from the question text. On CV-Bench, the forest is trained to predict which option is correct. Tables 4 and 5 list representative features for each question type; the implementation also derives closely related frequency and ratio features.

The reported implementation uses scikit-learn forests with no maximum depth: 100 trees for classifiers and 200 for regressors; rankings barely change across forests of 200 to 2,000 trees and depth 10, 20, or unlimited (Appendix D.5).

Avoiding label leakage in TsT-RF features. A TsT-RF feature must be computable without the evaluated question’s own answer. Statistics fit on the training folds, such as how often an object is the answer to a question type, meet this requirement; a lookup keyed on the held-out question’s gold answer does not, because it encodes the label. The features marked [‡] in Tabs. 4 and 5 are such lookups and are excluded from every TsT-RF score and automated selection we report. A simple check is label invariance: permuting the labels of the held-out fold must leave all of its feature values unchanged.

C.3. TsT-RF Results

Table 3 gives TsT-RF scores with random and grouped folds.

Table 3 | **TsT-RF cross-validation scores.** Features are label-conditioned (Sec. 3.3), so these are designer-side audit statistics. Baselines are for scale: uniform chance over each question’s options, averaged over questions, and VSI-Bench’s per-task frequency baseline [42]. Scores are averaged over questions (MC accuracy, plus NUM MRA for VSI-Bench). Grouped folds keep all questions about one scene (VSI-Bench) or image (CV-Bench) in the same fold. Features marked [‡] in Tabs. 4 and 5 are excluded. Dataset revisions: CV-Bench bc284db, VSI-Bench bc96b17.

Benchmark	Uniform chance	Frequency baseline	TsT-RF CV score	
			Random folds	Grouped folds
CV-Bench	41.8	–	56.1	55.7
VSI-Bench	–	34.0	43.0	38.8

D. Debiasing Details

D.1. IBP Details

In Algorithm 1, SelectBatch($\cdot$) can remove the highest-scoring questions directly or balance categorical groups, always ranking by the chosen per-question score ($s(x)$ or $\Delta s(x)$).

D.2. MMMU Negative-Control Results

Tab. 6 gives the counts and scores behind Sec. 4.3.

D.3. VSI-Bench-Debiased

VSI-Bench-Debiased uses hand-written per-type filters and hand-set budgets. All ten types remain, but removal rates range from 2.3% (easy relative direction) to 85.7% (medium relative direction; Tab. 7). The resulting shift in question-type mix is a limitation of the pilot and motivates an automated successor.

The pilot widens the gap more than random removal at the same rate (Tab. 8). Across seven models (Tab. 9), it widens most for Cambrian-S and the fine-tuned LLaVA-Video, the two with the highest original blind scores. It also widens for InternVL3-9B (11.1 to 13.2), whose language model is not from the Qwen family, so the widening is not limited to Qwen-based models. The gap barely moves for models with the lowest original blind scores.

Table 4 | **Representative VSI-Bench TsT-RF features by question type.** Features come from the question text and, for MC questions, the answer options; none uses visual input. [‡]Keyed on the held-out question’s gold answer; excluded from all reported TsT-RF scores and automated selections.

Task	Format	Feature Type	Feature Name	Description
Object Counting	NUM	Categorical	object	Object category
		Numerical	obj_count obj_val_log_mean obj_val_log_std global_mean_log global_std_log	Count of this object Mean of log gold values Std dev of log gold values Global mean (log space) Global std dev (log space)
Object Size Estimation	NUM	Categorical	object	Object category
		Numerical	obj_count obj_val_log_mean obj_val_log_std obj_freq_score obj_val_log_ratio	Count of this object Mean of log gold sizes Std dev of log gold sizes Min-max scaled object count One minus scaled std/mean of log sizes
Object Abs. Distance	NUM	Categorical	object_pair	Sorted pair of objects
		Numerical	pair_count pair_val_mean_log pair_val_std_log	Count of this pair Mean of log distances Std dev of log distances
Room Size Estimation	NUM	Numerical	global_mean_log global_std_log	Global mean (log space) Global std dev (log space)
Object Rel. Distance	MC	Categorical	target_object object_{i}	Category of target object Object category of option i , $\forall i \in [4]$
		Numerical	opt_{i}_obj_freq opt_{i}_pair_freq [‡] opt_{i}_ord_pair_freq [‡] max_opt_obj_freq max_opt_pair_freq [‡] max_opt_ord_pair_freq [‡]	Frequency of option i ’s object being correct Frequency of the (target, gold object) pair being correct Ordered-pair version of the above Max option frequency Max of the pair frequencies Max of the ordered-pair frequencies
Object Rel. Direction	MC	Categorical	difficulty positioning_object orienting_object querying_object	Subtype (easy/med/hard) Object the viewer stands by Object the viewer faces Query target object
		Numerical	obj_freq_score	Combined frequency of all 3 objects being in a question
Route Planning	MC	Categorical	beginning_object facing_object target_object opt_{i}	Starting position object Initial orientation object Destination object Route string option i
		Numerical	num_steps num_choices opt_{i}_freq_score obj_freq_score	Number of steps per route option (equal across options) Number of MC options Route option i frequency Combined frequency of all 3 objects being in a question
Appearance Order	MC	Categorical	opt_seq_{i}	Object sequence for option i : $[o_1^i, o_2^i, \dots, o_n^i]$
		Numerical	seq_{i}_pos_score	Sum of frequencies of object o_j^i appearing in position j in a gold sequence, $\forall j \in [n]$
			seq_{i}_adj_pair_score	Sum of frequencies of adjacent pairs o_j^i, o_{j+1}^i in a gold sequence, $\forall j \in [n-1]$
			seq_{i}_comb_pair_score	Sum of frequencies of all combinatorial pairs o_j^i, o_k^i in a gold sequence, $\forall j, k \in [n], j < k$
			seq_{i}_score	Average of the above three scores for option i

Table 5 | **Representative CV-Bench TsT-RF features by question type.** All tasks are MC; no feature uses visual input. [‡]Keyed on the held-out question’s gold answer; excluded from all reported TsT-RF scores.

Task	Feature Type	Feature Name	Description
2D Count	Categorical	object	Object category
	Numerical	n_options	Number of MC choices
		obj_count	Count of this object
		obj_val_log_mean	Mean of log gold values
		obj_val_log_std	Std dev of log gold values
		global_mean_log	Global mean (log space)
		global_std_log	Global std dev (log space)
2D Spatial Relation	Numerical	opt_{i}_dist_from_obj_mean	Log-space distance of option i from the object’s mean count
		opt_{i}_dist_from_global_mean	Log-space distance of option i from the global mean count
	Categorical	object_{i}	Category of object i , $\forall i \in [2]$
	Numerical	n_options	Number of MC choices
		pair_freq_score	Frequency of object pair in questions
		pair_answer_freq_score [‡]	Training-fold frequency of the gold answer for this pair
		is_majority_answer [‡]	Whether the gold answer is the training-fold majority
		question_length	Question character length
3D Depth	Numerical	contains_{dir}	Contains spatial keyword dir , for $dir \in \{\text{left, right, above, below, front, behind}\}$
		spatial_keyword_count	Total spatial direction keywords
	Categorical	object_{i}	Category of object i , $\forall i \in [2]$
	Numerical	n_options	Number of MC choices
		pair_freq_score	Frequency of object pair in questions
		pair_answer_freq_score [‡]	Training-fold frequency of the gold answer for this pair
3D Distance	Categorical	object_{i}	Category of object i , $\forall i \in [2]$
	Numerical	n_options	Number of MC choices
		pair_freq_score	Frequency of object pair in questions
	Numerical	pair_answer_freq_score [‡]	Training-fold frequency of the gold answer for this pair

Table 6 | **Pruning MMMU by $\Delta s(x)$ behaves like random removal.** LLaVA-OneVision-7B accuracy averaged over questions; gaps use unrounded scores.

Subset	n	Vis.	Blind	Gap
Original	900	49.4	42.8	6.7
$\Delta s(x)$ -ranked removal	800	47.9	40.4	7.5
Random removal	800	–	–	6.69 ± 0.58

Random removal: mean $\pm$ s.d. over 1,000 seeds; range 4.62 to 8.75.

Table 7 | **VSI-Bench-Debiased removes questions unevenly across types.**

Question Type	Format	Original	Removed	Removed (%)	Kept
Object size	NUM	953	600	63.0	353
Absolute distance	NUM	834	400	48.0	434
Relative distance	MC	710	400	56.3	310
Appearance order	MC	618	300	48.5	318
Object counting	NUM	565	314	55.6	251
Relative direction (med.)	MC	378	324	85.7	54
Relative direction (hard)	MC	373	257	68.9	116
Relative direction (easy)	MC	217	5	2.3	212
Room size	NUM	288	88	30.6	200
Route planning	MC	194	80	41.2	114
Total		5,130	2,768	54.0	2,362

Table 8 | **Random removal at the pilot’s rate barely changes the gap.** Gap on the full set, on the debiased set, and after randomly removing 54% of questions (mean $\pm$ s.d. over 10 seeds).

Model	Full gap	Pilot	Random
LLaVA-Video-7B + FT	12.4	16.7	12.4 $\pm$ 0.6
Cambrian-S	19.2	26.0	19.4 $\pm$ 0.8

Table 9 | **The gap widens most for Cambrian-S and the fine-tuned LLaVA-Video-7B.** V and B are vision and blind scores (MC accuracy and NUM MRA averaged over questions; NUM answers normalized for number words and units), sorted by original blind score. Gaps use unrounded scores. Cambrian-S is a pre-release checkpoint; base LLaVA-Video-7B comes from a separate evaluation, so its scores differ slightly from Tab. 2.

Model	LLM backbone	Original			Debiased			Δ gap
		V	B	Gap	V	B	Gap	
Cambrian-S	Qwen2.5	66.1	46.9	19.2	58.3	32.3	26.0	+6.8
LLaVA-Video-7B + FT	Qwen2	57.1	44.7	12.4	48.7	32.0	16.7	+4.3
InternVL3-9B	InternLM3	43.6	32.5	11.1	41.5	28.2	13.2	+2.2
InternVL2.5-26B	InternLM2.5	45.3	32.5	12.8	38.4	25.1	13.3	+0.5
InternVL2.5-8B	InternLM2.5	44.0	31.6	12.4	37.5	24.2	13.3	+0.9
LLaVA-OneVision-7B	Qwen2	35.9	27.0	9.0	29.6	21.2	8.5	-0.5
LLaVA-Video-7B	Qwen2	37.0	25.9	11.2	31.9	20.2	11.6	+0.5

D.4. Automated IBP on VSI-Bench

Per-format IBP ranks MC and NUM questions separately because their score scales differ; the per-type variants are defined in Sec. 4.2. Table 10 gives the resulting gaps.

Table 10 | **Automated per-type IBP widens the gap less than the pilot, and per-format IBP leaves it unchanged.** Gap of LLaVA-Video-7B fine-tuned on VSI-Train-10k, averaged over questions. Random removal at the same rates: 12.2 (per-format), 12.2 (per-type, uniform), 12.4 (per-type, pilot budgets).

Selection	Removed (%)	Kept	Gap
Original	0.0	5,130	12.4
Per-format, $B=1000$	19.5	4,130	12.4
Per-type, uniform	54.0	2,360	14.1
Per-type, pilot budgets (seeds 1, 42)	54.0	2,362	14.5, 14.7
Pilot	54.0	2,362	16.7

For per-format IBP, mean $s(x)$ falls steadily for MC and NUM as the budget grows (Tab. 11), although at $B=1000$ the gap does not move (Tab. 10). We use $B=1000$ to balance bias reduction against coverage.

D.5. Diagnostic Dependence

TsT-LLM and TsT-RF remove largely different questions (Tab. 12): their removed sets have Jaccard 0.060 (68 shared of 1,132 in the union). Part of the difference may come from their ranking scores, $\Delta s(x)$ for TsT-LLM and $s(x)$ for TsT-RF.

Replacing Qwen2-7B with LLaMA-3-8B in TsT-LLM (same folds and seed, native chat templates, no room-size questions) gives $\Delta s(x)$ rankings with Spearman 0.58 and 59.7% top-1,000 overlap. Within TsT-RF, forests of 200 to 2,000 trees and depth 10, 20, or unlimited give pairwise Spearman at least 0.990 and top-1,000 Jaccard at least 0.873. Rankings thus change substantially across base models but little across forest settings; there is no single correct pruned set.

Table 11 | **Mean $s(x)$ falls steadily as the per-format IBP budget grows.** Mean TsT-RF $s(x)$ after pruning.

B	Removed (%)	Mean $s(x)$	
		MC	NUM
200	3.9	0.337	0.421
500	9.7	0.308	0.385
1,000	19.5	0.270	0.329
1,500	29.2	0.246	0.270
2,000	39.0	0.219	0.210
2,500	48.7	0.200	0.138

Table 12 | **TsT-LLM and TsT-RF remove most question types at very different rates.** Removal rates (%) by question type. Budgets are 200 for TsT-LLM and 1,000 for TsT-RF. At a matched budget of 200, Jaccard is 0.050.

Question type	TsT-LLM	TsT-RF
Appearance order	15.5	40.1
Object counting	15.9	15.2
Object size	0.4	31.2
Absolute distance	0.2	12.2

E. TsT-RF Interpretability Analysis

We illustrate TsT-RF on VSI-Bench’s object-size questions (`object_size_estimation`). Feature importances explain TsT-RF; they are not a faithful explanation of how an MLLM answers.

Setup. The question template is “What is the length of the longest dimension (length, width, or height) of the {object}, measured in centimeters?” Each question gets six features: the object category and five statistics of that category computed on the training folds only (`obj_count`, `obj_freq_score`, `obj_val_log_mean`, `obj_val_log_std`, `obj_val_log_ratio`).

Results. With 5-fold cross-validation, TsT-RF reaches 61.4% MRA, well above VSI-Bench’s 29.9% frequency baseline for this task [42]. Gini importance concentrates on one feature, `obj_val_log_mean` (the category’s mean log size), at 0.968; the other five together contribute less than 0.04. The forest thus mostly predicts each category’s typical size.

Root Cause. Categories with very low size variation, such as dishwashers, beds, and washers, can be answered from category statistics alone; high-variation categories such as ceiling lights and radiators are much harder to predict this way (Tab. 13).

Implication for Benchmark Design. To close this shortcut, designers can remove questions about low-variance categories or sample instances so that sizes vary within each category.

Table 13 | **Categories with near-constant sizes are easy to exploit.** Size statistics for selected categories. CV is the coefficient of variation (standard deviation over mean).

Object	Mean (cm)	Std Dev (cm)	CV
<i>Low-variance objects (easily exploitable)</i>			
Dishwasher	90.4	3.4	0.037
Bed	216.1	17.2	0.080
Washer	87.1	5.0	0.058
Kettle	23.8	1.5	0.062
Mouse	11.6	1.1	0.091
<i>High-variance objects (harder to exploit)</i>			
Ceiling Light	71.8	55.8	0.778
Radiator	146.3	110.4	0.755